

Sos un Ingeniero de Software con años de experiencia y gran conocedor de buenas prácticas.

Quiero crear un repositorio con una estructura profesional. Debe tener un Changelog, archivos para git y docker, contributing.md.

Dentro de la carpeta Adopta (repositorio), debe haber tres directorios, uno para el frontend llamado Mobile, uno para el back llamado Server y uno intermedio llamado Shared (para compartir types entre front y back).

Tené en cuenta que las tecnologías a utilizar en el proyecto serán: React Native con Expo, Typescript, Zustand, Node con Express entre otras posibles.

Teniendo en cuenta las tecnologías genera la estructura de carpetas y dame los comandos necesarios para construirla, quiero que generes la estructura de los tres subdirectorios, vos elegí como dividis cada una (por ejemplo en frontend dividir carpetas por features o componentes, vos elegí según lo que sea mas profesional en las tres carpetas). Agregá archivos que creas necesarios y explicame porque.

Tené en cuenta el alcance del sistema para dar tu respuesta. Podes hacerme preguntas antes de tu respuesta final si lo crees necesario

[Mostrar menos](#)

Ejecutó 5 comandos, visualizó un archivo >

Perfecto, leí el alcance completo. Tengo todo lo que necesito para darte una respuesta sólida. El sistema **Adopta** es una app móvil de adopción de mascotas con un panel de administrador, integración con WhatsApp, geolocalización, y un roadmap claro de 3 entregas. No necesito hacerte preguntas previas, puedo tomar decisiones fundamentadas.

Voy a construir la estructura completa ahora.

Ejecutó 2 comandos >

Ahora voy a poblar los archivos clave con contenido real:

Ejecutó 10 comandos, leyó un archivo >

Acá está el repositorio completo. Te explico las decisiones de diseño:

Decisiones de arquitectura

Organización general: Feature-based (en los tres paquetes)

Elegí organizar **por features** (`animals`, `requests`, `admin`) en lugar de por tipo de archivo, porque el alcance ya define entidades claras y esta estructura escala mejor hacia la E2 y E3 (donde se agregan `shelters`, `adoptionFlow`, etc.). Agregar una feature nueva no toca archivos de otras.

`Shared/` — El contrato entre capas

Contiene únicamente TypeScript puro, sin dependencias de runtime. Los tipos como `Animal`, `AdoptionRequest` y `ApiResponse<T>` se definen acá una sola vez y los consumen tanto Mobile como Server. Esto evita la drift de tipos (que el back devuelva un campo y el front espere otro nombre).

- `animal.types.ts` ya incluye `AnimalWithDistance` pensando en RF-02 (geolocalización) y preparado para E2.
 - `api.types.ts` define `PaginatedResponse<T>` porque la E2 incorpora filtros avanzados y el listado va a necesitar paginación.
-

`Mobile/` — Feature-based con barrel index

Cada feature (`animals`, `admin`) tiene su propia carpeta con: `components/`, `screens/`, `store/`, `hooks/`, `services/`. Esta separación obedece a un principio: **el store de Zustand, los hooks y el servicio HTTP de una feature no deberían conocer los de otra.**

- `src/shared/` (dentro de Mobile) guarda lo transversal: componentes de UI genéricos (`Button`, `Card`, `EmptyState`), hooks como `useWhatsApp` (RF-06) y `usePermissions` (RF-04).
 - `src/theme/` centraliza colores, tipografía y espaciado para poder mantener consistencia visual sin valores hardcodeados.
 - `src/navigation/` está separado porque el grafo de rutas cambia mucho entre entregas y no pertenece a ninguna feature en particular.
-

`Server/` — Arquitectura en capas por feature

Cada feature sigue el patrón `routes → controller → service → repository`:

- **routes:** define los endpoints Express y llama al controller.
- **controller:** maneja request/response, valida con DTOs (usando Zod).
- **service:** lógica de negocio pura, sin saber nada de HTTP.

- **repository**: acceso a datos. Abstraer acá facilita cambiar ORM o base de datos.
- **dto/**: objetos de transferencia tipados para validar el body entrante.

`src/shared/middleware/` tiene `errorHandler` centralizado (ya implementado), `validateRequest` (para correr Zod antes de llegar al controller) y `uploadMiddleware` (para las imágenes de los animales, RF-03).

Archivos que quizás no esperabas

Archivo	Por qué existe
<code>.gitattributes</code>	Normaliza <code>LF</code> en todos los SO; evita conflictos de line endings en equipo
<code>.editorconfig</code>	Consistencia de indentación sin depender del IDE
<code>docker-compose.dev.yml</code>	Dev con hot-reload montado como volumen; <code>docker-compose.yml</code> para producción
<code>Server/uploads/.gitkeep</code>	Trackea la carpeta en git pero ignora su contenido (las imágenes subidas)
<code>database/migrations/</code> y <code>seeds/</code>	Carpetas preparadas para cuando incorpores un ORM (TypeORM/Prisma) en E2
<code>CI workflow</code>	Corre type-check, lint y tests en cada PR contra <code>main</code> o <code>develop</code>
<code>PULL_REQUEST_TEMPLATE</code>	Obliga a quienes contribuyen a declarar el tipo de cambio y hacer el checklist



Adopta
ZIP

Descargar

El repositorio ya lo tengo creado, como hago para crear toda la estructura que me diste a ese repo

Tenés dos opciones dependiendo de cómo tengas el repo:

Opción A — Clonar el repo vacío y copiar la estructura

```
bash

# 1. Clonar tu repo (reemplazá con tu URL)
git clone https://github.com/tu-usuario/adopta.git
cd adopta

# 2. Descomprimir el zip que te generé adentro del repo
unzip /ruta/al/Adopta.zip -d /tmp/adopta_temp

# 3. Copiar todo el contenido (sin la carpeta raíz Adopta/)
cp -r /tmp/adopta_temp/Adopta/. .

# 4. Primer commit
git add .
git commit -m "chore: initial project structure"
git push origin main
```

Opción B — El repo ya tiene commits/archivos

```
bash

# 1. Clonar tu repo
git clone https://github.com/tu-usuario/adopta.git
cd adopta

# 2. Crear una rama para no tocar main directamente
git checkout -b chore/initial-structure

# 3. Descomprimir y copiar
unzip /ruta/al/Adopta.zip -d /tmp/adopta_temp
cp -r /tmp/adopta_temp/Adopta/. .

# 4. Commitear y abrir PR
git add .
git commit -m "chore: initial project structure"
git push origin chore/initial-structure
```

Si estás en Windows

En PowerShell, en lugar del `cp -r`:

powershell

Descomprimir

```
Expand-Archive -Path "C:\ruta\Adopta.zip" -DestinationPath "C:\temp\adopta_temp"
```

Copiar contenido al repo

```
Copy-Item -Path "C:\temp\adopta_temp\Adopta\*" -Destination "." -Recurse -Force
```

Lo más común es la Opción A. El truco clave es el punto final en `cp -r` `/tmp/adopta_temp/Adopta/. .` — copia el *contenido* de la carpeta Adopta al directorio actual, sin crear una subcarpeta extra.

Dame comandos de terminal para crear las carpetas y archivos necesarios. No hace falta que tengan contenido

bash

Carpetas

```
mkdir -p .github/ISSUE_TEMPLATE .github/workflows
```

```
mkdir -p Shared/src/types Shared/src/constants Shared/src/utils
```

```
mkdir -p Mobile/src/features/animals/{components,screens,store,hooks,services}
```

```
mkdir -p Mobile/src/features/admin/{components,screens,store,hooks,services}
```

```
mkdir -p Mobile/src/shared/components/{ui,layout}
```

```
mkdir -p Mobile/src/shared/{hooks,services,utils}
```

```
mkdir -p Mobile/src/{navigation,config,theme}
```

```
mkdir -p Mobile/{assets/images,assets/fonts,__tests__}
```

```
mkdir -p Server/src/features/animals/{controller,service,repository,dto,routes}
```

```
mkdir -p Server/src/features/requests/{controller,service,repository,dto,routes}
```

```
mkdir -p Server/src/shared/{middleware,utils,errors}
```

```
mkdir -p Server/src/{config,database/migrations,database/seeds}
```

```
mkdir -p Server/{uploads,__tests__/features/animals,__tests__/features/requests}
```

Archivos raíz

```
touch .gitignore .gitattributes .editorconfig .env.example
```

```
touch docker-compose.yml docker-compose.dev.yml
```

```
touch CHANGELOG.md CONTRIBUTING.md README.md LICENSE
```

```
touch .github/PULL_REQUEST_TEMPLATE.md
```

```
touch .github/ISSUE_TEMPLATE/bug_report.md .github/ISSUE_TEMPLATE/feature_request.md
```

```
touch .github/workflows/ci.yml
```

Shared

```
touch Shared/package.json Shared/tsconfig.json Shared/README.md Shared/src/index
touch Shared/src/types/animal.types.ts Shared/src/types/user.types.ts
touch Shared/src/types/request.types.ts Shared/src/types/api.types.ts
touch Shared/src/constants/endpoints.ts Shared/src/constants/config.ts
touch Shared/src/utils/validation.ts
```

Mobile

```
touch Mobile/App.tsx Mobile/app.json Mobile/babel.config.js
touch Mobile/package.json Mobile/tsconfig.json Mobile/Dockerfile Mobile/.env.exa
touch Mobile/src/config/env.ts
touch Mobile/src/features/animals/components/AnimalCard.tsx
touch Mobile/src/features/animals/components/AnimalList.tsx
touch Mobile/src/features/animals/components/AnimalImageCarousel.tsx
touch Mobile/src/features/animals/components/InterestButton.tsx
touch Mobile/src/features/animals/screens/AnimalListScreen.tsx
touch Mobile/src/features/animals/screens/AnimalDetailScreen.tsx
touch Mobile/src/features/animals/store/animalStore.ts
touch Mobile/src/features/animals/hooks/useAnimals.ts
touch Mobile/src/features/animals/hooks/useGeolocation.ts
touch Mobile/src/features/animals/services/animalService.ts
touch Mobile/src/features/animals/index.ts
touch Mobile/src/features/admin/components/AnimalForm.tsx
touch Mobile/src/features/admin/components/RequestCard.tsx
touch Mobile/src/features/admin/screens/AdminDashboardScreen.tsx
touch Mobile/src/features/admin/screens/CreateAnimalScreen.tsx
touch Mobile/src/features/admin/screens/EditAnimalScreen.tsx
touch Mobile/src/features/admin/screens/RequestsScreen.tsx
touch Mobile/src/features/admin/store/adminStore.ts
touch Mobile/src/features/admin/hooks/useAdmin.ts
touch Mobile/src/features/admin/services/adminService.ts
touch Mobile/src/features/admin/index.ts
touch Mobile/src/shared/components/ui/Button.tsx
touch Mobile/src/shared/components/ui/Card.tsx
touch Mobile/src/shared/components/ui/EmptyState.tsx
touch Mobile/src/shared/components/ui/LoadingSpinner.tsx
touch Mobile/src/shared/components/ui/ErrorMessage.tsx
touch Mobile/src/shared/components/layout/Screen.tsx
touch Mobile/src/shared/hooks/useWhatsApp.ts
touch Mobile/src/shared/hooks/usePermissions.ts
touch Mobile/src/shared/services/api.ts
touch Mobile/src/shared/services/storage.ts
touch Mobile/src/shared/utils/formatters.ts
touch Mobile/src/shared/utils/distance.ts
touch Mobile/src/navigation/RootNavigator.tsx
```

```
touch Mobile/src/navigation/UserNavigator.tsx
touch Mobile/src/navigation/AdminNavigator.tsx
touch Mobile/src/navigation/types.ts
touch Mobile/src/theme/colors.ts
touch Mobile/src/theme/typography.ts
touch Mobile/src/theme/spacing.ts
touch Mobile/src/theme/index.ts
touch Mobile/__tests__/AnimalCard.test.tsx

# Server
touch Server/package.json Server/tsconfig.json Server/jest.config.ts
touch Server/Dockerfile Server/.dockerignore Server/.env.example Server/README.m
touch Server/src/app.ts Server/src/server.ts
touch Server/src/config/env.ts Server/src/config/database.ts Server/src/config/s
touch Server/src/database/migrations/.gitkeep Server/src/database/seeds/.gitkeep
touch Server/src/features/animals/controller/animal.controller.ts
touch Server/src/features/animals/service/animal.service.ts
touch Server/src/features/animals/repository/animal.repository.ts
touch Server/src/features/animals/dto/create-animal.dto.ts
touch Server/src/features/animals/dto/update-animal.dto.ts
touch Server/src/features/animals/routes/animal.routes.ts
touch Server/src/features/animals/index.ts
touch Server/src/features/requests/controller/request.controller.ts
touch Server/src/features/requests/service/request.service.ts
touch Server/src/features/requests/repository/request.repository.ts
touch Server/src/features/requests/dto/create-request.dto.ts
touch Server/src/features/requests/routes/request.routes.ts
touch Server/src/features/requests/index.ts
touch Server/src/shared/middleware/errorHandler.ts
touch Server/src/shared/middleware/validateRequest.ts
touch Server/src/shared/middleware/uploadMiddleware.ts
touch Server/src/shared/utils/logger.ts
touch Server/src/shared/utils/response.ts
touch Server/src/shared/errors/AppError.ts
touch Server/src/shared/errors/HttpError.ts
touch Server/uploads/.gitkeep
touch Server/__tests__/features/animals/animal.service.test.ts
touch Server/__tests__/features/requests/request.service.test.ts
```

Corré todo junto desde la raíz del repo. Al terminar hacés:

```
bash

git add .
git commit -m "chore: initial project structure"
```

git push

